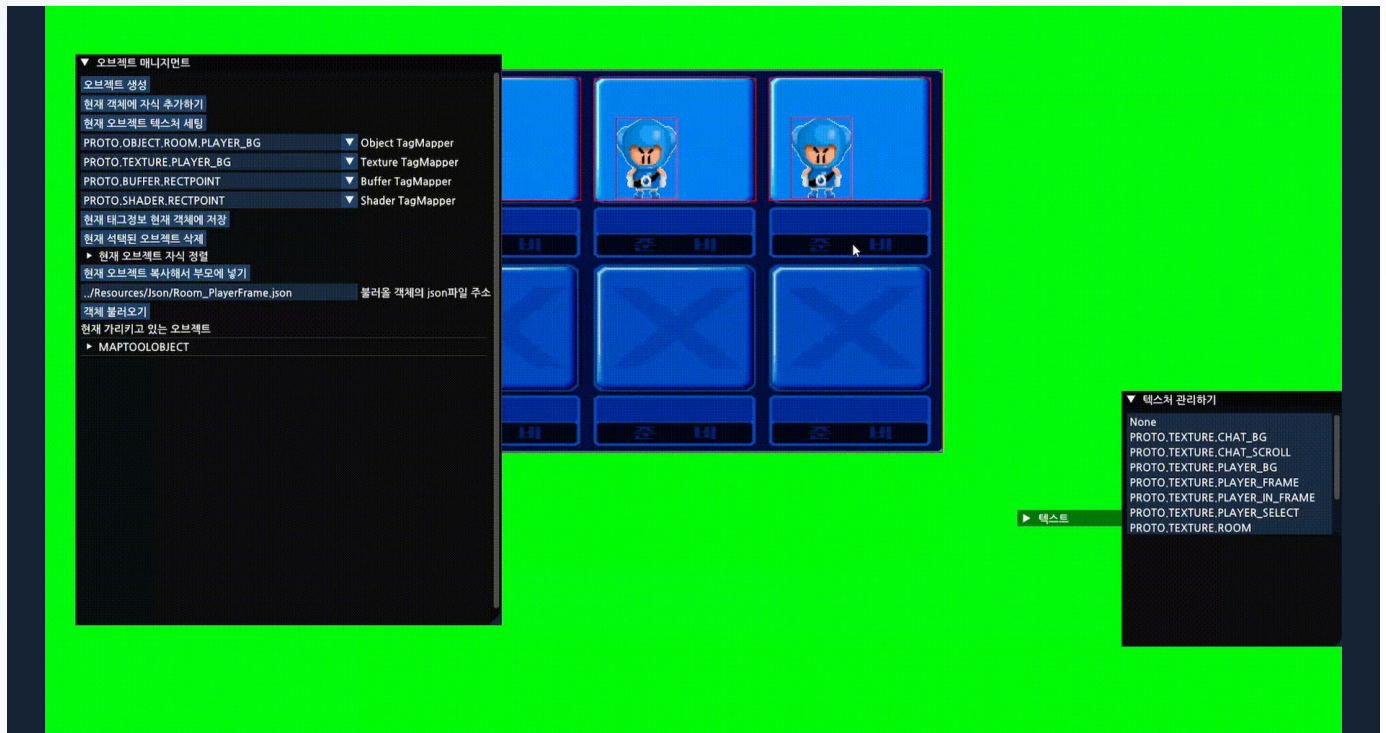


PROJECT PORTFOLIO

크레이지 아케이드 채팅 서버

ImGui UI 배치 도구와 JSON 화면 재구성, TCP·IOCP 채팅 서버와 클라이언트를 구현한 개인 프로젝트다.



UI 도구 · 저장한 화면 다시 불러오기

프로젝트	개인 프로젝트
담당 역할	UI 배치 도구, JSON 직렬화, TCP 클라이언트, IOCP 채팅 서버
기술	C++, DirectX, HLSL, ImGui, JSON, TCP/IP, IOCP
기간	2025.07.08 ~ 2025.07.29

<https://yollo4179.github.io/projects/crazy-arcade-chat-server/>

내가 구현한 영역

UI 배치 도구, JSON 직렬화, TCP 클라이언트, IOCP 채팅 서버

크레이지 아케이드 스타일의 화면에서 UI를 직접 배치하고, 채팅과 캐릭터 정보를 여러 클라이언트에 전달하기 위해 만든 개인 작업이다. UI를 코드에 고정하지 않고 ImGui 도구에서 부모·자식 트리로 만들었다. 도구가 저장한 JSON을 클라이언트가 읽어 같은 구조의 화면을 구성한다.

채팅은 클라이언트의 TCP 송수신 스레드와 IOCP 서버를 연결했다. 사용자가 메시지나 캐릭터 정보를 보내면 서버의 워커 스레드가 완료 큐에서 수신 작업을 받아 프로토콜 코드에 따라 처리하고, 연결된 클라이언트에 전달한다.

담당 구현

- **UI 제작:** ImGui 객체 선택·복제, 부모·자식 계층, 크기·위치·텍스처·Z 순서 편집, 그리드 정렬.
- **화면 데이터:** UI 트리의 JSON 저장과 로드, `ObjectBuilderSystem`을 통한 클라이언트 화면 재구성.
- **네트워크:** TCP 클라이언트 연결·송수신, 채팅·캐릭터 정보 프로토콜, IOCP 서버의 완료 큐와 워커 스레드.

기술 구성

C++, DirectX, HLSL, ImGui, JSON, TCP/IP, IOCP

실행 화면과 구현 코드에 연결된 블로그·영상 링크는 PDF에서 클릭해 열 수 있습니다.

02 / 구현 목차

기능별 구현

01	부모·자식 UI를 배치하는 ImGui 도구	04
02	UI 트리를 JSON으로 저장하고 다시 조립하기	07
03	TCP 클라이언트와 IOCP 채팅 서버 연결	13

01 / UI 제작 도구

부모·자식 UI를 배치하는 ImGui 도구

프로토타입으로 만든 UI 객체를 도구에서 편집하고, 부모 패널의 패딩·간격·열 개수로 자식을 재배포한다.

[블로그 원문 ↗](#)

문제와 구현

ImGui 도구는 기본 UI 객체의 프로토타입을 복제해 도구 화면에 등록한다. 도구에서 UI를 선택하면 크기·위치·텍스처를 바꾸거나 자식 UI를 추가할 수 있다. 복제 객체는 오브젝트 매니저의 업데이트·렌더 준비 경로에도 등록된다. 선택한 UI를 클릭하면 Z 순서를 올려 다른 UI보다 뒤에 그려지지 않도록 했다.

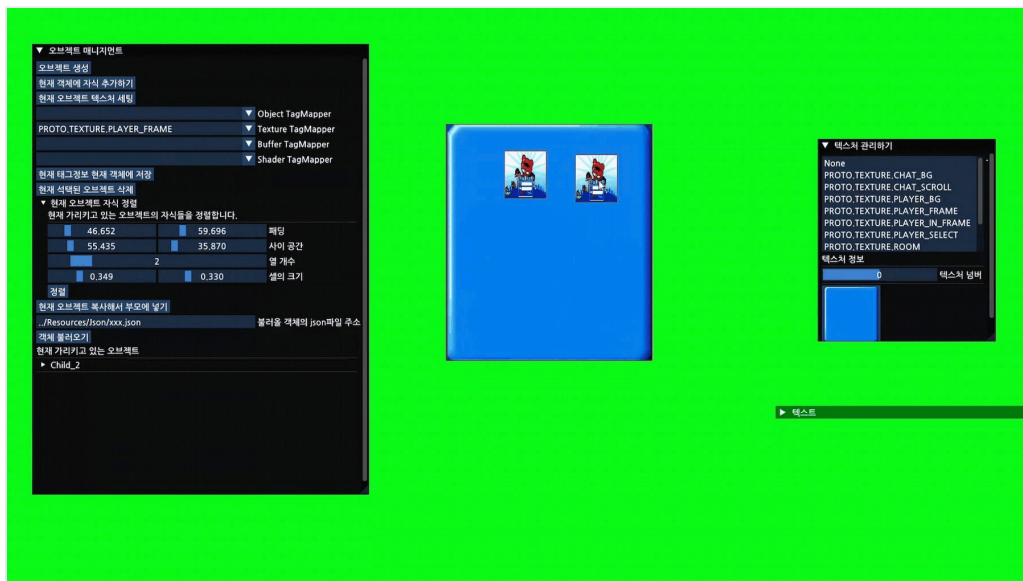
자식 UI의 위치는 부모가 이동하거나 크기가 바뀌어도 관계를 유지해야 한다. 도구는 자식의 부모 대비 크기와 부모 변환 행렬, 이동량을 조합해 최종 변환을 계산한다. 그리드 정렬 기능은 부모 패널의 크기에서 기준점을 구하고, 패딩·셀 크기·열 개수·셀 사이 간격으로 각 자식의 위치를 다시 계산한다. 정렬 기능을 실행하면 자식 UI의 변환과 크기가 한 번에 갱신된다.

TEXT

ImGui 배치 도구 → 부모·자식 UI 트리 → JSON 저장



클라이언트 화면 ← 재귀적 UI 조립 ← JSON 로드



그리드 배치 도구가 자식 UI의 크기, 패딩, 간격에 따라 위치를 조정하는 과정

선택한 UI의 텍스처 지정 · [영상 파일 열기](#)

선택한 UI의 렌더 순서 조정 · [영상 파일 열기](#)

정렬 계산의 책임

`CLevelMapTool::ReArrangeChildrenOf()`는 현재 선택한 부모 객체에 패딩, 자식 사이 간격, 열 개수, 셀 크기를 입력받는다. 정렬 명령은 부모의 `RearrangeChildGrid()`를 호출한다. 부모가 움직이거나 복제돼도 자식의 상대 배치가 유지되도록, 자식 추가와 복제는 부모 객체의 계층 API를 통해 처리한다. 객체를 직접 화면 좌표 목록으로만 저장했다면 부모 이동 때 자식 좌표를 모두 다시 계산해야 했다.

부모·자식 UI를 배치하는 ImGui 도구 · 계속

구현 코드

Engine/Private/GameObject.cpp 발췌

```
CPP

void CGameObject::RearrangeChildGrid(_float2 Padding, _float2 Space, _float2 CellSize, _uint NumColumns)
{
    RearrangeUISystem::RearrangeGridUI(shared_from_this(), m_Children, Padding, Space, CellSize, NumColumns);
}
```

셀 크기와 열 개수로 자식 배치

부모 패널의 크기와 패딩으로 시작점을 계산한다. 자식 인덱스를 열 개수로 나눈 나머지가 0이면 다음 행으로 이동하고, 그 외에는 X 방향으로 셀 크기와 간격을 더한다.

Engine/Public/UIRearrangeSystem.h 발췌

```
CPP

void RearrangeGridUI(
    const shared_ptr<CGameObject>&ParentObject,
    const vector<shared_ptr<CGameObject>>& ChildrenPtr,
    _float2 Padding,
    _float2 Space,
    _float2 CellSize,
    _uint NumColumns =1
)
{
    _float4 vPanelInfo = ParentObject->ComputeCollisionRect();
    _float2 PanelSize = {
        (vPanelInfo.y - vPanelInfo.x) ,
        (vPanelInfo.w - vPanelInfo.z)
    };
    _float2 vPivot = _float2{
        -1.f* PanelSize.x * 0.5f ,
        1.f* PanelSize.y * 0.5f
    };
    Padding.y *= -1.f;
    _vector vInitPosition = {
        vPivot.x + Padding.x+ (CellSize.x* PanelSize.x * 0.5f),
        vPivot.y + Padding.y- (CellSize.y* PanelSize.y * 0.5f),
        0.f,
        1.f
    };
}
```

부모·자식 UI를 배치하는 ImGui 도구 · 계속

CPP · 계속

```
};
_vector vCursor = vInitPosition;
_float SpaceY = {};
int iIndex = 0;
for (auto Child: ChildrenPtr)
{
    if ( 0 == (iIndex % NumColumns))
    {
        vCursor.m128_f32[0] = vInitPosition.m128_f32[0];
        if(iIndex) vCursor.m128_f32[1] -= (Space.y + (CellSize.y * PanelSize.y));
    }
    else
        vCursor.m128_f32[0] += (Space.x + (CellSize.x* PanelSize.x));
    Child->GetTransform()->SetState(CTransform::STATE_POSITION, vCursor);
    Child->GetTransform()->Scailing(_float3(CellSize.x, CellSize.y ,1.f));
    iIndex++;
}
}
```

02 / UI 제작 도구

UI 트리를 JSON으로 저장하고 다시 조립하기

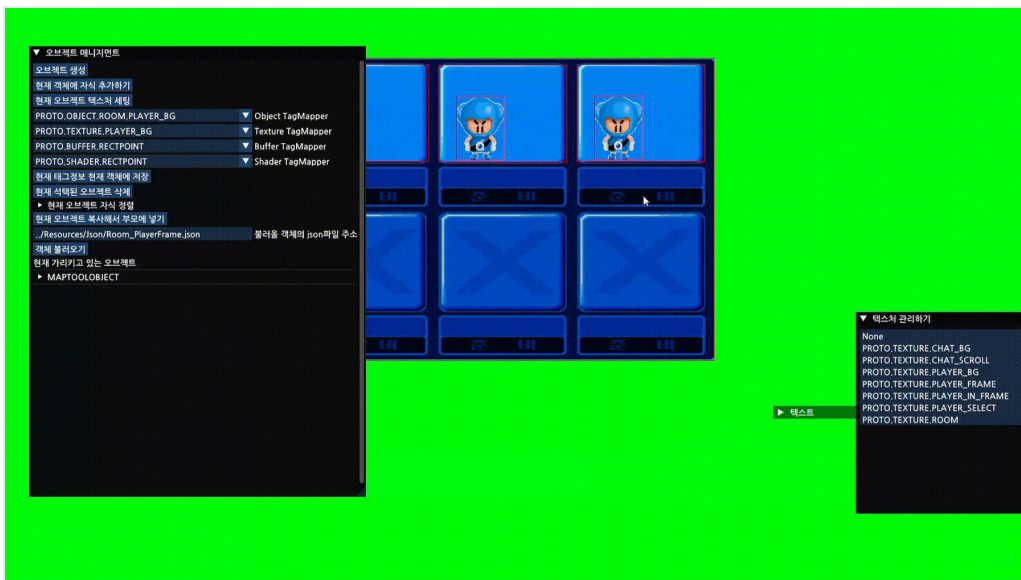
SRT_MATRIX와 컴포넌트 태그, CHILDREN을 저장한 뒤 ObjectBuilderSystem이 재귀적으로 UI를 만든다.

[블로그 원문 ↗](#)

문제와 구현

도구는 한 화면의 UI 세트를 JSON 객체로 저장한다. 각 노드에는 객체·텍스처·셰이더·버퍼의 태그와 레벨, 변환 행렬, Z 순서, 자식 노드 목록을 기록한다. 저장 함수는 부모 노드를 기록한 뒤 CHILDREN을 따라 자식 노드를 재귀적으로 저장한다. 불러올 때도 같은 트리를 순회한다.

클라이언트의 ObjectBuilderSystem::BuildObject는 JSON 노드의 OBJECT_TAG, TEXTURE_TAG, SHADER_TAG, BUFFER_TAG, SRT_MATRIX를 읽고 필요한 컴포넌트를 붙인다. CHILDREN 배열이 있으면 자식 객체를 만들고 같은 함수를 호출해 트리를 완성한다. 따라서 도구가 저장한 부모·자식 관계를 클라이언트 화면에서도 재구성할 수 있다.



저장된 UI 배치를 JSON에서 다시 불러오는 과정

```
CToolObject::TOOLOBJECTDESC CLevelMapTool::GetToolObjectDesc()
{
    CToolObject::TOOLOBJECTDESC tDesc = {};
    tDesc.fRotationPerSec = XMConvertToRadians(90.F);
    tDesc.fSpeedPerSec = 10.f;
    tDesc.ObjectPosition = { 0.f, 0.f, 1.f };
    tDesc.ObjectSize = { 100, 100, 1.f };
    tDesc._ZOrder = 10;
    tDesc._bSelected = false;
    tDesc._UIMode = UIMODE::UI_MOVABLE;
    tDesc._UIState = UISTATE::UI_RELEASED;

    return tDesc;
}
```

도구 객체의 기본 배치 속성

UI 트리를 JSON으로 저장하고 다시 조립하기 · 계속

```
shared_ptr<CToolObject> CLevelMapTool::CreateObject()
{
    shared_ptr<CGameObject> pUIObject;
    CToolObject::TOOLOBJECTDESC tDesc = { GetToolObjectDesc()};
    /*Add GameObject To Layer */
    m_pGameInstance.lock()->AddCopyObjectToLayer(
        (_uint)LEVEL_ID::LEVEL_MAPTOOL,
        L"PROTO.OBJECT.TOOLOBJECT",
        (_uint)LEVEL_ID::LEVEL_MAPTOOL,
        L"MAPTOOLOBJECT",
        &tDesc,
        &pUIObject);

    m_NowObject = dynamic_pointer_cast<CToolObject>(pUIObject);
    return m_NowObject;
}
```

HRESULT Engine::CGameInstance::AddCopyObjectToLayer(Engine::_uint iPrototypeLevel, const Engine::_wstring &PrototypeTag, Engine::_uint iLayerLevel, const Engine::_wstring &strLayerTag, void *pArg = (void *)nullptr, std::shared_ptr<...> * m_pGameObject = (std::shared_ptr<...> *)nullptr)

For Object Manager
+1개 오버로드
온라인 검색

프로토타입에서 도구 객체를 복제하는 코드

```
void CLevelMapTool::ClickCreateObjectButton()
{
    if (ImGui::Button("오브젝트 생성")) {
        vecObjects.push_back(dynamic_pointer_cast<CToolObject>(CreateObject()));
    }
}
```

ImGui에서 복제한 객체를 등록하는 코드

```
const _float4x4& CToolObject::ComputeWorldMatrixAsChild()
{
    _matrix matChild = m_pTransformCom->GetWorldMatrix();
    _matrix matChildSR = matChild;
    matChildSR.r[3] = _vector{ 0.f,0.f,0.f,1.f };

    _matrix matChildT = XMMatrixIdentity();
    matChildT.r[3] = matChild.r[3];
    _matrix retMat = {};

    retMat = matChildSR * m_pParent.lock()->GetChildWorldMatrix() * matChildT;
    //부모대비 나의 배율( 0~1 )*기준(부모) + 절대적 이동량
    XMStoreFloat4x4(
        &m_ChildWorldMatrix,
        retMat
    );
    return m_ChildWorldMatrix;
}
```

부모와 자식 UI의 변환 행렬

UI 트리를 JSON으로 저장하고 다시 조립하기 · 계속

```
void CLevelMapTool::AddChildToNowObject()
{
    if(ImGui::Button("현재 객체에 자식 추가하기"))
    if (m_NowObject.get())
    {
        CToolObject::TOOLOBJECTDESC tDesc{ GetToolObjectDesc() };
        shared_ptr<CGameObject> pChildPtr = dynamic_pointer_cast<CGameObject>(
            m_pGameInstance.lock()->Clone_Prototype(
                PROTOTYPE::PROTO_OBJ,
                (_uint)LEVEL_ID::LEVEL_MAPTOOL,
                L"PROTO.OBJECT.TOOLOBJECT",
                &tDesc
            )
        );
        m_NowObject->AddChildObject(pChildPtr);
    }
}
```

자식 UI를 부모에 추가하는 코드

```
void CLevelMapTool::ReArrangeChildrenOf()
{
    if (ImGui::TreeNode("현재 오브젝트 자식 정렬")) {
        if (m_NowObject) {
            ImGui::Text("현재 가리키고 있는 오브젝트의 자식들을 정렬합니다.");
            ImGui::Separator();
            static _float2 Space = { 1.f,1.f };
            static _float2 Padding = { 1.f,1.f };
            static _uint Columns = { 1 };
            static _float2 CellSize = { 10.f,10.f };

            ImGui::SliderFloat2("패딩", (_float*)&Padding, 0.f, 300.f);
            ImGui::SliderFloat2("사이 공간", (_float*)&Space, 0.f, 300.f);
            ImGui::SliderInt("열 개수", (_int*)&Columns, 1, 10);
            ImGui::SliderFloat2("셀의 크기", (_float*)&CellSize, 0.1f, 1.f);
            if(ImGui::Button("정렬"))
                m_NowObject->RearrangeChildGrid(Padding, Space, CellSize, Columns);
        }
        ImGui::TreePop();
    }
}
```

그리드 정렬의 위치 계산 코드

```
void CGameObject::RearrangeChildGrid(_float2 Padding, _float2 Space, _float2 CellSize, _uint NumColumns)
{
    RearrangeUISystem::RearrangeGridUI(shared_from_this(), m_Children, Padding, Space, CellSize, NumColumns);
}
```

그리드 정렬의 자식 UI 적용 코드

UI 트리를 JSON으로 저장하고 다시 조립하기 · 계속

```

"ATLASINFO": [
],
"BUFFER_LEVEL": 0,
"BUFFER_TAG": "PROTO_BUFFER_RECTPOINT",
"CHILDREN": [
],
"ATLASINFO": [
0.0,
0.0,
0.0,
0.0
],
"BUFFER_LEVEL": 0,
"BUFFER_TAG": "PROTO_BUFFER_RECTPOINT",
"CHILDREN": [],
"OBJECT_LEVEL": 5,
"OBJECT_TAG": "PROTO_OBJECT_ROOM_SELECT_BUTTON",
"SHADER_LEVEL": 0,
"SHADER_TAG": "PROTO_SHADER_RECTPOINT",
"SRT_MATRIX": {
"LOOK": [
0.0,
0.0,
1.0,
0.0
],
"POSITION": [
-262.32623291015625,
2.424774169921875,
0.0,
1.0
],
"RIGHT": [
0.2129999952316284,
0.0,
0.0,
0.0
],
"UP": [
0.0,
0.2029999941587448,
0.0,
0.0
]
},
"TEXTURE_LEVEL": 5,
"TEXTURE_TAG": "PROTO_TEXTURE_PLAYER_SELECT"

```

UI 트리의 JSON 저장 형식

```

#ifdef _DEBUG
void CGameObject::Inspector()
{
    static char g_Address[MAX_PATH] = "../Resources/Json/";
    static char g_FileName[MAX_PATH] = "xxx";
    if (ImGui::TreeNode(m_pGameInstance->Wbcs2Mbc(m_strName).c_str())) {
        /*Components 먼저 */
        /*Texture + Transform + Collider + (Buffer)*/
        ImGui::InputText("제이슨 파일 이름(JSON확장자 빼고)", g_FileName, MAX_PATH * sizeof(char));
        if (ImGui::Button("제이슨 데이터 해당 주소로 저장"))
        {
            try {
                ofstream Out;
                json jsonData = {};
                WriteJsonFile(jsonData);
                Out.exceptions(std::ofstream::failbit | std::ofstream::badbit);
                string GameAddress = string(g_Address) + string(g_FileName) + string(".json");

                Out.open(GameAddress.c_str());
                Out << jsonData.dump(4); Out.close();
            }
            catch(const std::ios_base::failure& e){
                e;
                MSG_BOX("저장 에러 발생");
            }
        }

        for (auto Component : m_Components)
        {
            if(Component.second)
                Component.second->Inspector();
        }
        ImGui::TreePop();
    }
}
#endif // _DEBUG

```

저장 버튼에서 직렬화 함수를 호출하는 코드

UI 트리를 JSON으로 저장하고 다시 조립하기 · 계속

```
void CGameObject::WriteJsonFile(json& j)
{
    {
        j["SRT_MATRIX"] = *(m_pTransformCom->GetWorldMatrixPtr());
    }
    j["OBJECT_TAG"] = WBCS2MBCS(m_ProtoObjTag);
    j["TEXTURE_TAG"] = WBCS2MBCS(m_PprotoTextureTag);
    j["SHADER_TAG"] = WBCS2MBCS(m_ProtoShaderTag);
    j["BUFFER_TAG"] = WBCS2MBCS(m_ProtoBufferTag);

    j["OBJECT_LEVEL"] = m_ProtoObjLevel;
    j["TEXTURE_LEVEL"] = m_ProtoTextureLevel;
    j["SHADER_LEVEL"] = m_ProtoShaderLevel;
    j["BUFFER_LEVEL"] = m_ProtoBufferLevel;
    j["ATLASINFO"] = m_AtlasInfo;

    json Jsonchildren = json::array();
    for (auto Child : m_Children)
    {
        json Node = {};
        Child->WriteJsonFile(Node);
        Jsonchildren.push_back(Node);
    }
    j["CHILDREN"] = std::move(Jsonchildren);
    return;
}
```

자식 UI를 재귀적으로 저장하는 코드

```
void CLevelMapTool::LoadObject()
{
    static char jsonPath[MAX_PATH] = "../Resources/Json/xxx.json";

    ImGui::InputText("불러올 객체의 json파일 주소", jsonPath, sizeof(char)*MAX_PATH);
    if(ImGui::Button("객체 불러오기"))
    {
        json ObjectInfo = {};
        ifstream In;

        try {
            In.exceptions(std::ofstream::failbit | std::ofstream::badbit);
            In.open(jsonPath);
            In >> ObjectInfo;
        }
        catch (const std::ios_base::failure& e) {
            MSG_BOX("저장 에러 발생");
            e;
        }
        In.close();
        /*객체 생성*/
        shared_ptr<CGameObject> pUIObject;
        CToolObject::TOOLOBJECTDESC tDesc = { GetToolObjectDesc() };
        m_pGameInstance->lock()->AddCopyObjectToLayer((uint)LEVEL_ID::LEVEL_MAPTOOL, L"PROTO.OBJECT.TOOLOBJECT", (uint)0, tDesc, pUIObject);
        vecObjects.push_back(dynamic_pointer_cast<CToolObject>(pUIObject));
        m_NowObject = dynamic_pointer_cast<CToolObject>(pUIObject);
        ObjectBuilder(ObjectInfo, pUIObject);
    }
}
```

UI 데이터 파일을 선택해 불러오는 코드

불러오기 실패 처리의 한계: LoadObject()는 읽기 실패에도 "저장 에러 발생"을 표시하고 객체 생성을 계속한다. 로드 실패 메시지로 수정하고 예외 처리 뒤 종료해야 한다. std::ofstream::failbit와 badbit는 공통 기반 ios_base의 플래그이므로 값 자체는 오류가 아니다. std::ios_base::failbit | std::ios_base::badbit로 표기하면 혼동을 줄일 수 있다.

UI 트리를 JSON으로 저장하고 다시 조립하기 · 계속

```
void CLevelMapTool::ObjectBuilder(json& node, shared_ptr<CGameObject> pGameObject)
{
    XMVECTOR matSRT = {}; string ObjTag = {}; string TextureTag = {}; string ShaderTag = {}; string BufferTag = {};
    _uint ObjLevel = {}; _uint TextureLevel = {}; _uint ShaderLevel = {}; _uint BufferLevel = {}; _float4 AtlasInfo = {};
    node.at("SRT_MATRIX").get_to(matSRT);
    node.at("OBJECT_TAG").get_to(ObjTag);
    node.at("OBJECT_LEVEL").get_to(ObjLevel);
    node.at("TEXTURE_TAG").get_to(TextureTag);
    node.at("TEXTURE_LEVEL").get_to(TextureLevel);
    node.at("SHADER_TAG").get_to(ShaderTag);
    node.at("SHADER_LEVEL").get_to(BufferLevel);
    node.at("BUFFER_TAG").get_to(BufferTag);
    node.at("BUFFER_LEVEL").get_to(BufferLevel);
    node.at("ATLASINFO").get_to(AtlasInfo);
    pGameObject->SetAtlasInfo(AtlasInfo);
    _wstring wTexTag = MBCS2WBCS(TextureTag);
    _wstring wBufTag = MBCS2WBCS(BufferTag);
    _wstring wShaTag = MBCS2WBCS(ShaderTag);
    _wstring wObjTag = MBCS2WBCS(ObjTag);
    pGameObject->SetProtoObjTagInfo(wObjTag, ObjLevel);
    pGameObject->SetProtoTextureInfo(wTexTag, TextureLevel);
    pGameObject->SetProtoBufferInfo(wBufTag, BufferLevel);
    pGameObject->SetProtoShaderInfo(wShaTag, ShaderLevel);
    pGameObject->MakeComponentsFromTagInToolLevel();
    pGameObject->GetTransform()->SetWorld(XMLoadFloat4x4(&matSRT));
    if (node.contains("CHILDREN") && node["CHILDREN"].is_array())
    {
        for (json& child : node["CHILDREN"])
        {
            CToolObject::TOOLOBJECTDESC tDesc = { GetToolObjectDesc() };
            /*오브젝트 먼저 생성*/
            shared_ptr<CGameObject> pChild = dynamic_pointer_cast<CGameObject>(m_pGameInstance.lock()->Clone_Prototype(PROTOTYPE::PROTO_
            pGameObject->AddChildObject(pChild); //자식으로 추가해놓고 다음에 바꾼다 .
            ObjectBuilder(child, pChild);
        }
    }
}
```

JSON 자식 노드를 재귀적으로 읽는 코드

저장 형식과 복원 순서

CGameObject의 저장 경로는 SRT_MATRIX, 객체·텍스처·셰이더·버퍼 태그, ATLASINFO, CHILDREN을 노드에 기록한다. 맵툴의 ObjectBuilder()는 태그를 먼저 읽어 컴포넌트를 구성하고 월드 행렬을 적용한 뒤 CHILDREN을 순회한다. 클라이언트의 ObjectBuilderSystem도 같은 키를 사용해 화면용 객체를 다시 만든다. 저장 형식의 키가 도구와 게임 사이의 계약이다.

클라이언트 조립 코드는 자식 행렬과 부모 행렬의 관계도 처리한다. 화면이 다시 불러와졌을 때 부모만 맞고 자식이 어긋나면 JSON의 좌표 값뿐 아니라 부모·자식 행렬을 어느 시점에 결합하는지도 확인해야 한다.

구현 코드

Client/Private/ObjectBuilder.cpp 발췌

```
CPP

if (NODE.contains("CHILDREN") && NODE["CHILDREN"].is_array())
{
    for (json& child : NODE["CHILDREN"])
    {
        string ObjectTag = {};
        _uint ObjectLevel = {};
        _float4x4 ChildMatrix = {};
        child["OBJECT_TAG"].get_to(ObjectTag);
        child["OBJECT_LEVEL"].get_to(ObjectLevel);
        shared_ptr<CGameObject> pChildNodePtr;
        child["SRT_MATRIX"].get_to(ChildMatrix);
        pChildNodePtr = dynamic_pointer_cast<CGameObject>(m_pGameInstance.lock()->Clone_Prototype(PROTOTYPE::PROTO_
OBJ, ObjectLevel, MBCS2WBCS(ObjectTag), &tDesc));
        BuildObject(child, pChildNodePtr);
        NodePtr->AddChildObject(pChildNodePtr);
        pChildNodePtr->GetTransform()->SetWorld(XMLoadFloat4x4(&ChildMatrix));
        pChildNodePtr->ComputeWorldMatrixAsChild();
    }
}
```

03 / 채팅 서버

TCP 클라이언트와 IOCP 채팅 서버 연결

REQ_LOGIN, SND_MESSAGE를 서버가 처리하고 ACK_LOGIN, BROAD_CAST_ALL을 각 클라이언트에 전달한다.

[블로그 원문 ↗](#)

문제와 구현

클라이언트는 서버 IP 주소와 포트에 TCP 소켓으로 연결한다. 연결이 성립하면 메인 스레드는 렌더링을 계속하고, 별도의 송신-수신 스레드가 네트워크 통신을 맡는다. 사용자가 채팅 입력을 끝내거나 캐릭터를 바꾸면 클라이언트는 프로토콜 코드와 데이터 길이, 실제 내용을 연속된 바이트 배열로 서버에 보낸다. 수신 스레드는 서버 데이터의 코드를 읽고 채팅 내용은 채팅 상자에 추가하며, 캐릭터 변경은 해당 클라이언트의 화면 상태에 반영한다.

서버의 리스너 소켓은 주소와 포트를 바인딩하고 새 연결을 받는다. 워커 스레드는 IOCP 완료 큐에서 수신 완료 작업을 꺼내 프로토콜 코드에 따라 데이터를 처리한다. 서버가 변경 정보를 다른 클라이언트에 전달하면, 각 클라이언트의 수신 스레드가 자기 화면에 같은 결과를 반영한다. TCP는 채팅에 필요한 데이터 도달 순서와 재전송을 제공한다는 점을 고려해 선택했다.

TEXT

```

클라이언트 A — 채팅·상태 전송 → TCP 서버
클라이언트 B ← 변경 정보 전달 — TCP 서버
                                ↳ IOCP 완료 큐 → 워커 스레드

```

채팅 서버와 클라이언트 시연 · 영상 파일 열기

```
uint ChattingSocketManager::AcquireSocket(_ustring HOST_IP, _ustring PORT_NUMBER)
{
    WSADATA wsa = { 0 };
    if (::WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        MSG_BOX("ERROR: 윈속 초기화 실패");
        return 0;
    }

    m_hSocket = ::socket(AF_INET, SOCK_STREAM, 0);
    if (m_hSocket == INVALID_SOCKET)
    {
        MSG_BOX("ERROR: 리스닝 소켓 생성 실패");
        return 0;
    }

    char HOST[MAX_PATH] = {};
    char PORT[MAX_PATH] = {};

    strcpy_s(HOST, MIBCS2MBCS(HOST_IP).c_str(), "//도메인");
    strcpy_s(PORT, MIBCS2MBCS(PORT_NUMBER).c_str(), "");
    //포트번호인 줄과 다들영문으로 제공받은 경우인 1p(mis)로 대명 포트(대명 포트 사이트가 제공)→25500(서버방문)

    /*제일먼저 TCP로 처리→설정 정보 tcp + IPv4 */
    addrinfo hints{};
    hints.ai_family = AF_INET; // IPv4
    hints.ai_socktype = SOCK_STREAM; // TCP
    hints.ai_protocol = IPPROTO_TCP;
    addrinfo* res = nullptr;

    //포트 정보 제공
    /* 서버 포트 제공*/
    SOCKADDR_IN svraddr = {};
    svraddr.sin_family = AF_INET;
    svraddr.sin_port = htons(25000); /*서버는 25000에 연결할, 해당 포트 닫았고 서버 코드가 요구할 것*/
    svraddr.sin_addr.S_un.S_addr = getaddrinfo(HOST, PORT, &hints, &res); /*서버의 아이피 주소 DNS to IP */

    /*연결 시도*/
    if (!connect(
        m_hSocket,
        (res->ai_addr),
        static_cast<int>(res->ai_addrlen)) == SOCKET_ERROR)
    {
        puts("ERROR: 서버에 연결할 수 없습니다.");
        return 0;
    }

    return m_hSocket;
}
```

서버 주소와 포트로 연결하는 코드

```
private:
    friend unsigned int WINAPI ThreadListening(void* pArg);
    friend unsigned int WINAPI ThreadSending(void* pArg);
```

클라이언트의 송수신 스레드 진입점

TCP 클라이언트와 IOCP 채팅 서버 연결 · 계속

```

BEGIN(Client)
...
unsigned int WINAPI ThreadListening(void* pArg) {

    ChattingSocketManager* pGlobalInstance = reinterpret_cast<ChattingSocketManager*>(pArg);
    pGlobalInstance->RecvLoop();
    return 0;

};

unsigned int WINAPI ThreadSending(void* pArg)
{
    ChattingSocketManager* GlobalInstance = reinterpret_cast<ChattingSocketManager*>(pArg);
    GlobalInstance->RequestSessionID();
    GlobalInstance->SendLoop();

    return 0;
}
END

```

송신 스레드 구성 코드

```

HRESULT ChattingSocketManager::Initialize(const _wstring&& HOST_IP, const _wstring&& PORT_NUMBER, _bool isOnChattingLevel)
{
    m_isOnChattingLevel = isOnChattingLevel;
    //InitializeCriticalSection(&m_CriticalSection);
    if (0 == AcquireSocket(HOST_IP, PORT_NUMBER)) { //Connect까지 진행
        MSG_BOX("인원 파악");
        return E_FAIL;
    }
    /*연결 완료 (나 일었다 소켓 클라) */
    m_hRecv =
        (HANDLE)_beginthreadex(
            nullptr,
            0,
            ThreadListening,
            this,
            0,
            nullptr
        );
    if (m_hRecv == INVALID_HANDLE_VALUE)
    {
        return E_FAIL;
    }

    m_hSend =
        (HANDLE)_beginthreadex(
            nullptr,
            0,
            ThreadSending,
            this,
            0,
            nullptr
        );
    CloseHandle(m_hSend);
    CloseHandle(m_hRecv); // UsageCount 2-> 1 종료시 0 -> 커널 오브젝트 0

    return S_OK;
}

```

수신 스레드 구성 코드

TCP 클라이언트와 IOCP 채팅 서버 연결 · 계속

```

uint ChattingSocketManager::SendLoop()
{
    char    StreamBuffer[8192] = {}; //8KB
    while (true == m_IsOnChattingLevel)
    {
        if (true == m_bSendState && -1 != SessionManager::Get().SessionID )
        {
            BYTE PCK[sizeof(CHAT_LOG)+ sizeof(PROTOCOL_HEADER)] = {};
            BYTE* pSeeker = PCK;
            //Header
            PROTOCOL_HEADER ProtocolHeader = {(uint)PROTOCOL_CODE::SEND_MESSAGE , sizeof(CHAT_LOG)};
            memcpy(PCK, reinterpret_cast<const char*>(&ProtocolHeader), sizeof(PROTOCOL_HEADER));
            int Size = sizeof(PROTOCOL_HEADER);
            pSeeker += sizeof(PROTOCOL_HEADER);
            /* Payload */

            wstring Log = SessionManager::Get().AccountID + _wstring(L": ") + m_MyString;
            m_bSendState = false;

            if (!m_MyString.size()) Log = {};

            CHAT_LOG m_ChatLog = {};
            memcpy(m_ChatLog.Account, SessionManager::Get().AccountID.c_str(), sizeof(m_ChatLog.Account));
            m_ChatLog.CharacterType = SessionManager::Get().MyCharacter;
            memcpy(m_ChatLog.Message, Log.c_str(), sizeof(m_ChatLog.Message));
            m_ChatLog.SSESSION_ID = SessionManager::Get().SessionID;
            m_ChatLog.vColor = { 1.f,1.f,1.f,1.f };

            /*패킷 완성*/
            memcpy(pSeeker, &m_ChatLog, sizeof(CHAT_LOG));
            int Sign = ::send(m_hSocket, reinterpret_cast<const char*>(PCK), sizeof(PCK), 0);
            if (SOCKET_ERROR == Sign)
            {
                int A = WSAGetLastError();
                MSG_BOX("소켓 쓰기 에러");
            }
            m_MyString = {};
        }
    }
    /*클라이언트 종료 버튼 누르면 이거 호출해준다 // 좀비처럼 연결되어있는 것을 방지 */
    shutdown(m_hSocket, SD_BOTH);
    closesocket(m_hSocket); /* 연속 해제*/
    ::WSACleanup();
    /*여기서 끝났는지 맞는지 알다.*/
    return 0;
}

```

채팅과 캐릭터 변경 정보를 전송하는 코드

```

uint ChattingSocketManager::RecvLoop()

{
    char    StreamBuffer[8192] = {}; //8KB
    BYTE* pSeeker = (BYTE*)StreamBuffer;
    while (0 < ::recv(m_hSocket, StreamBuffer, sizeof(StreamBuffer), 0)) { //소켓 끊어지지만 않으면 뎀

        //Header 읽기

        PROTOCOL_HEADER Cmd;
        memcpy(&Cmd, reinterpret_cast<const char*>(StreamBuffer), sizeof(PROTOCOL_HEADER));
        pSeeker += sizeof(PROTOCOL_HEADER);

        switch (Cmd.Protocol)
        {

```

수신 데이터의 프로토콜 분기 코드

```

case (uint)PROTOCOL_CODE::BROAD_CAST_ALL:
{
    {
        CHAT_LOG ChatLog = {};
        memcpy(&ChatLog, pSeeker, sizeof(CHAT_LOG));

        if (ChattingLogs.size() >= CONST_MESSAGE_LIMIT) {
            ++DisplayIndex[0]; ++DisplayIndex[1];
        }
        if (strlen(ChatLog.Message))
            ChattingLogs.push_back( make_pair(ChatLog.Message, XMLoadFloat4(&ChatLog.vColor)));

        if (0 > ChatLog.SSESSION_ID)
        {
            MSG_BOX("NO");
            return 1;
        }

        shared_ptr<CPlayerFrame> pFrame = dynamic_pointer_cast<CPlayerFrame>(
            m_pGameInstance->GetUIObjectByTag(L"PLAYERS_BACKGROUND")->FindChildByIndex(ChatLog.SSESSION_ID));
        pFrame->SetSessionID(ChatLog.SSESSION_ID);

        shared_ptr<CPlayerInFrame> pPlayerInFrame= dynamic_pointer_cast<CPlayerInFrame>(pFrame->FindChildByIndex(0));
        pPlayerInFrame->SetCharacter(ChatLog.CharacterType);

        shared_ptr<CPlayerNameInFrame> pPlayerNameInFrame = dynamic_pointer_cast<CPlayerNameInFrame>(pFrame->FindChildByIndex(1));
        pPlayerNameInFrame->SetAccountName(ChatLog.Account);
        pPlayerNameInFrame->Active();
    }

    break;
}

```

클라이언트 화면에 수신 내용을 반영하는 코드

TCP 클라이언트와 IOCP 채팅 서버 연결 · 계속

완료 큐에서 메시지를 화면까지

서버의 `PROTOCOL_HEADER`는 프로토콜 코드와 뒤따르는 데이터 크기를 담는다. `REQ_LOGIN`을 받으면 서버는 세션 ID를 `ACK_LOGIN`으로 돌려주고 플레이어 정보도 함께 보낸다. `SND_MESSAGE`를 받으면 `CHAT_LOG`에서 계정·캐릭터 정보를 갱신한 뒤 `BROADCAST_ALL` 패킷을 만든다. 서버는 패킷을 연결된 소켓에 전송하고, 클라이언트 수신 스레드는 채팅 상자와 캐릭터 표시를 갱신한다.

`WorkingThread()`는 `GetQueuedCompletionStatus()`에서 완료된 수신을 받고, 처리 뒤 `WSARecv()`를 다시 등록한다. 이 방식에서 IOCP는 수신 완료를 워커에 전달하는 역할이다. 이 코드의 브로드캐스트 송신은 별도 완료 큐에 올린 비동기 송신이 아니라 `send()` 호출이다. 또한 TCP의 한 번 수신에 항상 한 패킷과 일치하는 보장은 없으므로, 패킷 길이에 맞춘 누적 버퍼와 범위 검사는 이후 보강할 부분이다.

구현 코드

IOCP_ChatServer/IOCP_ChatServer.cpp 발췌

CPP

```
case (_uint)PROTOCOL_CODE::REQ_LOGIN:
{
    BYTE PCK[ sizeof(PROTOCOL_HEADER) + sizeof(AccountID) + sizeof(PROTOCOL_HEADER) + sizeof(g_PlayerInfo) ] = {};
    memset(PCK, 0, sizeof(PROTOCOL_HEADER) + sizeof(AccountID) + sizeof(PROTOCOL_HEADER) + sizeof(g_PlayerInfo));
    PROTOCOL_HEADER* pCmd = (PROTOCOL_HEADER*)PCK;
    pCmd->Protocol = { (_uint)PROTOCOL_CODE::ACK_LOGIN };
    pCmd->NextFileSize = { sizeof(AccountID) };
    AccountID* pSessionID = (AccountID*)(PCK + sizeof(PROTOCOL_HEADER));
    pSessionID->SESSION_ID = pSession->SESSION_ID;
    {
        PROTOCOL_HEADER* pCmd = (PROTOCOL_HEADER*)((BYTE*)pSessionID + sizeof(AccountID));
        pCmd->Protocol = { (_uint)PROTOCOL_CODE::SND_PLAYERS_INFO };
        pCmd->NextFileSize = { sizeof(g_PlayerInfo) };
        PLAYER_INFO* pPlayerInfo = (PLAYER_INFO*)((BYTE*)pCmd + sizeof(PROTOCOL_HEADER));
        memcpy(pPlayerInfo, &g_PlayerInfo[0], sizeof(g_PlayerInfo));
        ::send(pSession->hSocket, (const char*)PCK, sizeof(PROTOCOL_HEADER) + sizeof(AccountID)+ sizeof(PROTOCOL_HE
        ADER) + sizeof(g_PlayerInfo), 0);
    }
    break;
}
```